Encryption and Key Management

Explore how encryption and key management safeguard data in AWS cloud-native applications. Understand symmetric and asymmetric encryption, AWS KMS key types, and encryption models like server-side and client-side encryption. Learn to configure encryption for AWS services, manage keys securely, and enable cross-account key access for comprehensive data protection.

Data protection is as important in cloud-native applications as writing performant and reliable code. Securing that data is foundational, whether our application stores personal user data, financial transactions, or internal logs. Encryption ensures that data remains confidential and tamper-proof, even if unauthorized access occurs. In the AWS ecosystem, encryption is embedded deeply in services across computing, storage, and networking. However, as developers, we must actively configure and manage encryption and its associated keys to meet technical and compliance requirements.

Core encryption concepts in AWS

Ask

AWS supports both symmetric and asymmetric encryption:

- **Symmetric encryption:** This method uses a single shared key to encrypt and decrypt the data. Anyone who has a copy of the key can encrypt and decrypt. The main challenge is securely sharing the key and preventing it from falling into the wrong hands.
- **Asymmetric encryption:** Uses a pair of keys, a public key to encrypt data, and a separate private key to decrypt it. Anyone can encrypt the data with the public key, but only the authorized individual can decrypt it using the private key.

Encryption is also categorised based on the state of the encrypted data, whether it's being stored or moved. Here's a simple breakdown:

- **Encryption at rest:** This refers to protecting data that is not moving on the client or server side, such as data stored on a hard drive, in a database, in an S3 bucket, or on a backup tape.
- **Encryption in transit:** This refers to protecting the data while it is actively moving from one place to another, such as across the internet or between different services within your network. We use HTTPS with an SSL/TLS certificate to secure the connection between the client and server.

AWS Key Management Service (KMS)

Encryption in AWS relies on the AWS Key Management Service (KMS). Instead of each service managing its encryption methods, AWS provides KMS as a centralized and secure service to create, store, and control access to the cryptographic keys that protect the data.

Ask

> **Note:** AWS provides multi-Region keys to create replicas of a primary key across regions. These are especially useful in applications with cross-region data replication or geographically distributed users.

AWS KMS is designed to protect the keys by **FIPS 140-2 Validated Hardware Security Modules (HSMs).** All KMS keys are generated and stored within HSMs, which are specialized, tamper-resistant hardware devices certified under the U.S. government's FIPS 140-2 standard. This provides a powerful guarantee that the plaintext key material can never leave the physical boundary of the HSM, not even to AWS employees. All cryptographic operations happen securely inside this hardened device, including encrypting or decrypting data.

Every key in KMS has a well-defined life cycle to further ensure that it is not accidentally exposed:

- **Enabled:** This is the default state for a new key. In this stage, the key is fully operational and can be used for any cryptographic operations its key policy allows, such as encrypting new data or decrypting existing data.
- **Disabled:** We can temporarily disable a key at any time. When a key is disabled, it becomes unusable for new cryptographic operations. No user or AWS service can use it to encrypt new data or decrypt existing data.
- **Pending deletion:** When we schedule a key for deletion, KMS enforces a mandatory waiting period of 7 to 30 days, which can be configured. During this time, the key is disabled and cannot be used. This waiting period acts as a crucial safety net, allowing users to cancel the deletion if they realize it was a mistake. Once the waiting period ends, the key and all its cryptographic material are permanently and irrecoverably destroyed.

Ask

Types of KMS keys

AWS KMS provides three types of cryptographic keys that can be categorized by who manages them. Understanding the difference between different types of keys is important for implementing the right security posture for applications.

Let's explore these in detail.

1. AWS-managed keys

These keys are created on the user's behalf when they use an AWS service that integrates with KMS for the first time. AWS-managed keys are easily identifiable by their alias, which follows the format `aws/<service-name>`.

For example, if we create an S3 bucket by selecting the "Server-side encryption with AWS Key Management Service keys (SSE-KMS)" option, AWS automatically gives the default option to create a new AWS-managed KMS key with the `aws/s3` alias.

We can view these keys and their policies in the KMS console, but we cannot change their policies or manage them directly. AWS manages its life cycle and rotation, which is mandatory every 365 days.

2. AWS-owned keys

AWS-managed keys are now legacy and are not created for new AWS services. AWS now uses AWS-owned keys to encrypt customer data by default. These keys are not in our AWS account. An AWS service owns and manages a collection of keys in its separate service account for use across multiple AWS customers.

The customer has no control over these keys. We cannot see them, manage their policies, or track their usage in CloudTrail. They are completely abstracted away from the user.

Ask

This is the default encryption model for many AWS services when we don't specify our key. For example, SQS queues are encrypted by default with an AWS-

owned key. It provides a baseline level of security without any configuration or cost.

3. Customer-managed keys (CMKs)

These are KMS keys created, owned, and managed completely by the customer within the AWS account. We can define the key policy, enable or disable it, and schedule it for deletion. We can also choose to rotate the keys automatically, manually, or not at all. In short, we have complete control over its life cycle.

We should use a customer-managed key when we need fine-grained control over who can access the key, as the access policy of AWS-managed keys can not be changed. This is the recommended key type for sensitive data.

Another important use case of CMK is sharing encrypted resources across accounts. As AWS-managed keys have their access policies managed entirely by AWS, they are designed to allow access from within the same account. We cannot edit these key policies to add another AWS account. This is a security feature to prevent accidental cross-account exposure.

> All keys are stored in a FIPS 140-2 validated hardware security module (HSM). A FIPS 140-2 validated HSM is a highly secure, tamper-resistant hardware device certified to meet strict government-approved standards for cryptographic security. Later, the encrypted key can be decrypted by KMS to decrypt the actual data. This approach reduces the load on KMS while maintaining high levels of security.

Encryption models in AWS

AWS supports two main encryption models: server-side encryption and cli side encryption. The key distinction lies in *who performs the encryption* and *where* it happens in the data flow.

Ask

Server-side encryption (SSE)

In server-side encryption, AWS handles encryption after receiving the data, but before it is written to disk in an AWS data center. It is a built-in process that manages encryption and decryption itself.

For example, when we upload an object to a service like Amazon S3, we send the data in its normal, unencrypted state, over a secure SSL/TLS connection. The AWS service that receives the data, encrypts it with a data key, and then stores that encrypted data. When we fetch the data, the service automatically decrypts it and returns the plaintext data.

The Python code below shows how we can upload objects to an S3 bucket using server-side encryption.

main.py Python

```python
import boto3

s3 = boto3.client('s3')
bucket_name = 'your-bucket-name'
file_path = 'example.txt'
object_key = 'encrypted-file.txt'

# KMS Key ID or alias (e.g., 'alias/my-kms-key')
kms_key_id = 'your-kms-key-id-or-alias'

# Upload with server-side encryption using SSE-KMS
s3.upload_file(
    Filename=file_path
```

Server-side encryption using Python

Server-side encryption is the most common method for protecting data at the cloud because it provides a powerful combination of security and simplicity.

Ask

There are three types of server-side encryption keys offered by AWS:

- **SSE-S3:** AWS fully manages the encryption keys for us, providing a simple, automatic baseline of security with no setup required.
- **SSE-KMS:** We use keys from the AWS KMS for centralized control. This allows us to manage permissions and track key usage with detailed audit trails in CloudTrail.
- **SSE-C:** We provide and manage our own encryption key for each object we upload, giving us full control over the keys but also full responsibility for securing and preventing their loss.

Client-side encryption (CSE)

In client-side encryption, we encrypt the data before sending it to AWS. The data is encrypted on our own application server or local machine before it leaves the environment. This provides the highest level of control and assurance, as unencrypted data never travels over the network or is seen by AWS.

Envelope encryption pattern

Client-side encryption almost always uses a secure pattern called envelope encryption. It's a layered approach that combines the strength of KMS with the efficiency of local symmetric encryption. The workflow is a simple, multi-step process:

1. **Request a data key:** The application makes an API call to AWS KMS to generate a unique data key.
2. **Return keys to the application:** The `GenerateDataKey` API returns two versions of this key: a plaintext version and a version encrypted by the master key in KMS.
3. **Encrypt data locally:** The application uses the plaintext key to encrypt our data, for example, a file, a database record, locally. The encrypted da then stored alongside the encrypted data key. The encrypted key is safe to store as it can only be decrypted by KMS with our specific permissions.

Ask

4. **Discard the plaintext key:** Immediately after encryption, the plaintext key is erased from memory. This is a critical security step.

Workflow of envelope encryption

The code below shows a simple example of this workflow using the AWS SDK for Python:

main.py SDK (Python)

```
import boto3
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad

kms = boto3.client('kms')
key_id = 'your-kms-key-id-or-alias'

# Step 1: Generate a data key
response = kms.generate_data_key(KeyId=key_id, KeySpec='AES_256')
plaintext_key = response['Plaintext']
encrypted_key = response['CiphertextBlob']

# Step 2: Encrypt some data locally
```

Ask

Client-side encryption using Python

This approach is ideal when we must retain full control over the encryption process from end to end such as:

- Operating under strict compliance regulations that require us to prove data is encrypted before it reaches the cloud.
- Ensuring data remains encrypted even if it's moved outside of the AWS environment.
- Supporting offline decryption requirements where an application may need to decrypt data without a live connection, using a securely managed key.

Performance for high-throughput applications

For applications that need to encrypt or decrypt a high volume of data, making a `GenerateDataKey` or `Decrypt` call to KMS for every operation can introduce latency and increase costs. To solve this, the AWS Encryption SDK provides a feature called **data key caching**. This approach securely caches a recently generated data key in memory, allowing it to be reused for multiple operations. Developers can define a cache time-to-live (TTL) and usage limits to balance performance with security, dramatically reducing API calls in workloads that repeatedly process data.

Encryption with AWS services

Encryption must often be explicitly configured for AWS services. As a developer, it's critical to understand how encryption integrates into common services:

- **Amazon RDS:** Supports encryption at rest using AWS KMS. Once enabled, encryption applies to all underlying storage, automated backups, read replicas, and snapshots. However, RDS encryption cannot be turned on after instance creation.

Ask

- **Amazon EBS:** Volumes can be encrypted at launch time using a specified KMS key. Snapshots created from encrypted volumes are also encrypted.
- **Amazon S3:** Use bucket policies to enforce server-side encryption by requiring requests to include the `x-amz-server-side-encryption` header. For example, add a condition to require `SSE-KMS` encryption with a specific key.

These integrations ensure that sensitive data is encrypted without requiring custom application logic.

Cross-account and cross-service encryption

Data often needs to be encrypted in one account (Account-A) and decrypted in another (Account-B). To make this work securely, we need to establish a two-way trust relationship. There are two main ways to grant cross-account and across-service key access.

Access through policies

This is a two-part process involving both the key owner and the key user:

1. **The key policy in the key owner's account:** The first step is for the owner of the key to explicitly grant permission to the other account. This is done by editing the key policy of the Customer Managed Key in KMS. This is the resource-based policy that opens the door.
2. **The IAM policy in the key's account:** The second step is to allow the user in the second account to access the key through the IAM policy. Just because "Account A" trusts "Account B" doesn't mean every user or service in "Account B" can automatically use the key. The administrator of "Account B" must now create an IAM policy that explicitly allows a specific user or role to perform actions on that key. Services like Lambda and EC2 operate an IAM role to get their permissions.

Ask

Cross account access through policies

This two-sided handshake ensures that access is never granted by accident. Both the key owner and the resource owner must agree.

Access through grants

While policies are great for static, long-term permissions, Grants are a more dynamic and temporary way to delegate access. A **grant** is a programmatic tool used to give temporary, fine-grained permissions to use a KMS key. An application or an AWS service that has permission can create a grant for another user or service. For example, many AWS services use grants behind the scenes to get temporary permission to use your KMS key on your behalf when performing an action, like encrypting an S3 object. This is more secure than giving the service broad, permanent access in its IAM policy.

Conclusion

In modern cloud-native development, encryption is a foundational pillar of application security. AWS makes encryption accessible through services like KMS, which centralizes key management and integrates with AWS services. By choosing the right encryption strategy and managing keys responsibly, we can

Ask